

When Your Safety Net Becomes a Trap

Cleaning Up Feature Flag Debt

Devopsdays Prague 2026

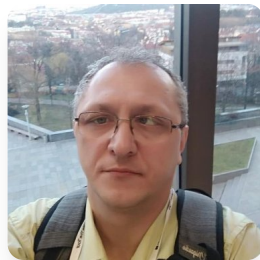
Oleksii Mudryk



Who am I

- 20+ years in software
development, deployment, technical leadership, platform work
- Always close to both the code
and the way it reaches production
- Years of finding, prioritising and paying down technical debt
and avoiding the next round
- What draws me to DevOps:
reliable delivery, understandable systems,
keeping long-lived software safe to change

```
manifesto:  
  rely:  
    on: mindset  
    over: skillset  
  balance:  
    - execution  
    - exploration  
  elude: |-  
    exertion  
    in favor of  
    excellence
```



How old is your oldest temporary flag?



How old is your oldest temporary flag?

This release.



How old is your oldest temporary flag?

This release.

Older than anything on the current roadmap.



How old is your oldest temporary flag?

This release.

Older than anything on the current roadmap.

Nobody knows.



On the way to operational excellence

Feature-flag cleanup is not really what this talk is about.

It's the epic that showed up on the way to something else.

*elude exertion
in favor of
excellence*

- Excellence isn't a burst of heroic effort – that's exertion, and it doesn't scale
- It's the boring, sustained, structural work most teams quietly skip
- Cleaning up a hundred flags, done properly, is exactly that kind of work

Flags that outlived their roadmap

Flags themselves were not new.

A hundred-plus of them *on a managed platform*

with a lifecycle and a bill attached

Empty Maintainer

Tags that meant something to someone, once

Last touched before half the current team joined



Flags that outlived their roadmap

Flags themselves were not new.

A hundred-plus of them *on a managed platform*

with a lifecycle and a bill attached

Empty Maintainer

Tags that meant something to someone, once

Last touched before half the current team joined

Twenty-plus years finding, prioritising and paying down technical debt –
in whatever shape it happened to take that year

rely: on: mindset, over: skillset



Why LaunchDarkly

Why LaunchDarkly

Two questions, really.

Why LaunchDarkly

Two questions, really.

...why it is called that

Why LaunchDarkly

Two questions, really.

...why it is called that

...why it holds at scale

- Flag rules stream to the SDK
and are cached in your process
- `variation()` evaluates locally –
no network call per flag
- Which is why **billions of evaluations** on one flag
is ordinary

Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.
It is deployed, it is running, nobody can see it.

...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.
It is deployed, it is running, nobody can see it.



...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.

It is deployed, it is running, nobody can see it.

Deploy and release stop being the same event.



...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

Why LaunchDarkly

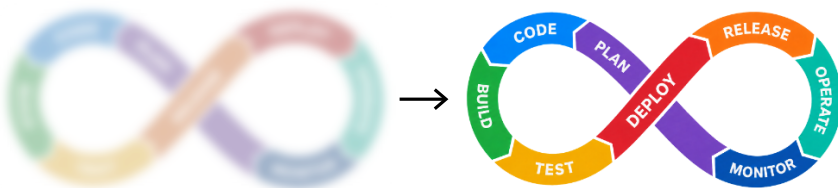
Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.

It is deployed, it is running, nobody can see it.

Deploy and release stop being the same event.



So do feature flags reverse the classical order?

...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.

It is deployed, it is running, nobody can see it.

Deploy and release stop being the same event.



Not really – release names two moments:

- The **version** is still released before deploy
- The **feature** is released later, during **operate**

...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

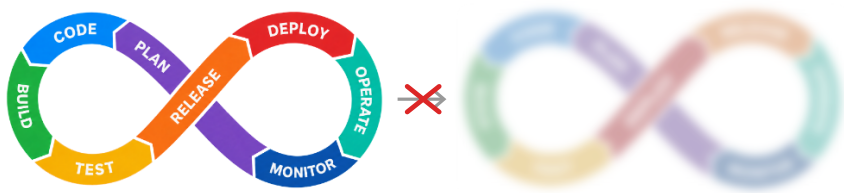
Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off.
It is deployed, it is running, nobody can see it.

Deploy and release stop being the same event.



Not really – release names two moments:

- The version is still released before deploy
- The feature is released later, during operate

...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

```
const enabled = client.variation(  
  'checkout-v2', context,  
  /* fallback */ false,  
)
```

launchdarkly.com/docs/sdk/concepts/client-side-server-side

Why LaunchDarkly

Two questions, really.

...why it is called that

Dark launch – ship the code to production with the feature switched off. It is deployed, it is running, nobody can see it.

Deploy and release stop being the same event.



Not really – release names two moments:

- The version is still released before deploy
- The feature is released later, during operate

...why it holds at scale

- Flag rules stream to the SDK and are cached in your process
- `variation()` evaluates locally – no network call per flag
- Which is why **billions of evaluations** on one flag is ordinary

```
const enabled = client.variation(  
  'checkout-v2', context,  
  /* fallback */ false,  
)
```

launchdarkly.com/docs/sdk/concepts/client-side-server-side

Why GitHub

- The flag platform knows what it was told:
keys, rules, rollout state
- It does not know where in the codebase a key is actually called
- That question lives in source
across however many repositories the organization has
- GitHub's code search can answer it
and it can search an entire organization, not just one repository

docs.github.com/en/search-github/github-code-search/understanding-github-code-search-syntax

The gap that starts the epic

- LaunchDarkly has its own **Code References** feature
genuinely useful, finds flag usage in code
- But it's wired **per repository**
every codebase needs its own scanner connected, on purpose, by someone
- At real org scale
many repositories, not all of them anyone's daily focus
there is no built-in **"show me every reference, everywhere"**

This is publicly documented vendor behaviour, not a complaint about the product.

launchdarkly.com/docs/home/flags/code-references

A permanent decision in a temporary costume

- Nobody schedules a project called "build a maze"
- Each flag, on its own, has a perfectly reasonable local explanation
- The maze is the *sum* of those explanations
not a mistake anyone made

This is the same shape every kind of technical debt takes.
Flags are just this year's costume for it.

A permanent decision in a temporary costume

- Nobody schedules a project called "build a maze"
- Each flag, on its own, has a perfectly reasonable local explanation
- The maze is the *sum* of those explanations
not a mistake anyone made

This is the same shape every kind of technical debt takes.
Flags are just this year's costume for it.

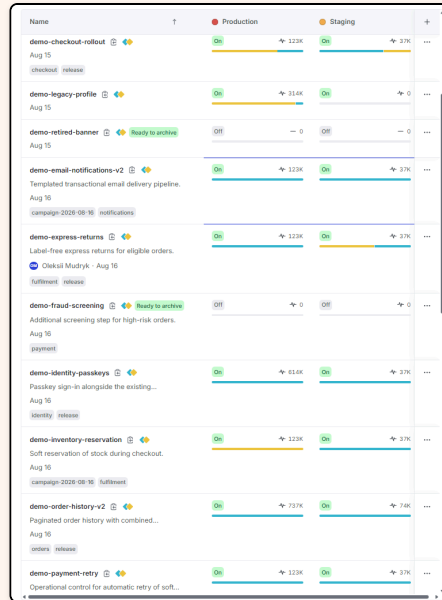
Blameless postmortems already gave us the words for this:
every step was locally reasonable, and the outcome still happened.

The bill arrives during reasoning, not during creation

- A stale flag is nearly free
as long as nobody looks at it
- It gets expensive under pressure
the moment someone has to answer:
which path is actually live?
- All of these pay this same tax:
incident diagnosis · onboarding
test coverage · ownership

The bill arrives during reasoning, not during creation

- A stale flag is nearly free as long as nobody looks at it
- It gets expensive under pressure the moment someone has to answer: which path is actually live?
- All of these pay this same tax: incident diagnosis · onboarding test coverage · ownership



Name	Production	Staging
demo-checkout-rollout	On 123K	On 37K
demo-legacy-profile	On 114K	On 0
demo-retired-banner	Ready to archive	Off 0
demo-email-notifications-v2	On 123K	On 37K
demo-express-returns	On 123K	On 37K
demo-fraud-screening	Ready to archive	Off 0
demo-identity-passkeys	On 114K	On 37K
demo-inventory-reservation	On 123K	On 37K
demo-order-history-v2	On 737K	On 74K
demo-payment-retry	On 123K	On 37K

The bill arrives during reasoning, not during creation

- A stale flag is nearly free as long as nobody looks at it
- It gets expensive under pressure the moment someone has to answer: which path is actually live?
- All of these pay this same tax: incident diagnosis · onboarding test coverage · ownership

A screenshot of a software deployment dashboard. The table lists various services and their status across Production and Staging environments. The services include demo-checkout-rollout, demo-legacy-profile, demo-retired-banner, demo-email-notifications-v2, demo-express-returns, demo-fraud-screening, demo-identity-passkeys, demo-inventory-reservation, demo-order-history-v2, and demo-payment-retry. Each service has a status indicator (On, Off, Ready to archive) and a percentage value (e.g., 123K, 37K, 31.6K, 0, 123K, 37K, 61.6K, 123K, 737K, 74K, 123K, 37K).

Name	Production	Staging
demo-checkout-rollout	On 123K	On 37K
demo-legacy-profile	On 31.6K	On
demo-retired-banner	Ready to archive	Off
demo-email-notifications-v2	On 123K	On 37K
demo-express-returns	On 123K	On 37K
demo-fraud-screening	Ready to archive	Off
demo-identity-passkeys	On 61.6K	On 37K
demo-inventory-reservation	On 123K	On 37K
demo-order-history-v2	On 737K	On 74K
demo-payment-retry	On 123K	On 37K

The bill arrives during reasoning, not during creation

- A stale flag is nearly free as long as nobody looks at it
- It gets expensive under pressure the moment someone has to answer: which path is actually live?
- All of these pay this same tax: incident diagnosis · onboarding test coverage · ownership



Name	Production	Staging
demo-checkout-rollout	On 123K	On 37K
demo-legacy-profile	On 314K	On
demo-retired-banner	Ready to archive	Off
demo-email-notifications-v2	On 123K	On 37K
demo-express-returns	On 123K	On 37K
demo-fraud-screening	Ready to archive	Off
demo-identity-passkeys	On 616K	On 37K
demo-inventory-reservation	On 123K	On 37K
demo-order-history-v2	On 737K	On 74K
demo-payment-retry	On 123K	On 37K

The dashboard is not the system

- The flag platform knows **configuration and evaluations**
- The repository knows **references**
where a key is actually called
- Deployment history knows **which versions are still running right now**
- People know the exception nobody wrote down
and it usually reaches paper only in an incident review

Safe cleanup needs all four.

The dashboard is only the easiest one to open.

Step 1 – Inventory from independent evidence

Not a clever deletion script.

A boring inventory that makes uncertainty visible.

flag inventory

joined by key with

org-wide code search

Step 1 – Inventory from independent evidence

Not a clever deletion script.

A boring inventory that makes uncertainty visible.

flag inventory

joined by key with

org-wide code search

May 2025 · one Excel script

one GitHub code search per flag key

moudrick.net/blog/ff-script

Step 1 – Inventory from independent evidence

Not a clever deletion script.

A boring inventory that makes uncertainty visible.

flag inventory

joined by key with

org-wide code search

May 2025 · one Excel script
one GitHub code search per flag key
moudrick.net/blog/ff-script

10

429 Too Many Requests
the flag where the first run stopped
docs.github.com/en/rest/search/search

Step 1 – Inventory from independent evidence

Not a clever deletion script.

A boring inventory that makes uncertainty visible.

flag inventory

joined by key with

org-wide code search

May 2025 · one Excel script
one GitHub code search per flag key
moudrick.net/blog/ff-script

The count was the easy part.

10

429 Too Many Requests
the flag where the first run stopped
docs.github.com/en/rest/search/search

X-RateLimit-Remaining
X-RateLimit-Reset
what it learned to wait on

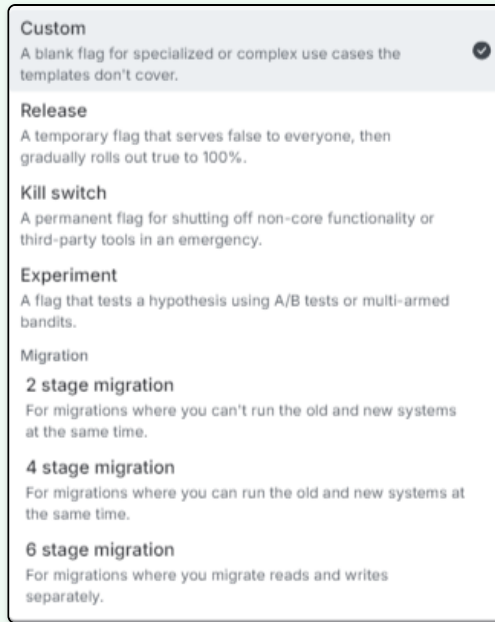
Step 2 – Classify on two axes, not one label

Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design



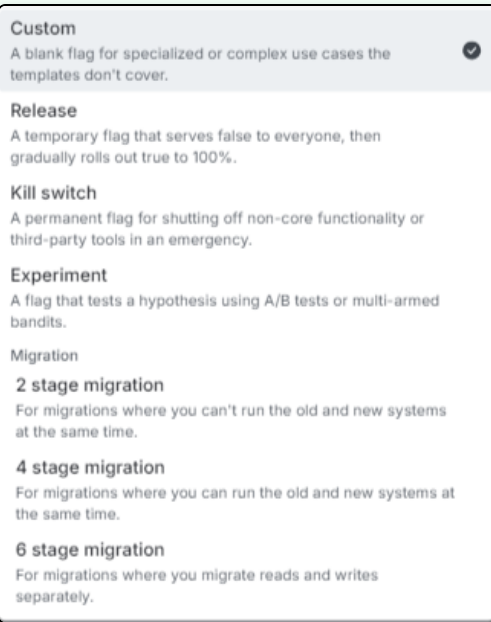
Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.



Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.

But intent does not tell you state.

the two axes – what live evaluations and cleanup show later

Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.

But intent does not tell you state.

the two axes – what live evaluations and cleanup show later

A flag has **two independent axes**:

- **Rollout state** – what is being served
not started · partial · 100%
rolled back · permanent control · unknown
- **Cleanup state** – what work remains
active · stale · dead candidate · zombie · unknown

Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.

But intent does not tell you state.

the two axes – what live evaluations and cleanup show later

A flag has **two independent axes**:

- **Rollout state** – what is being served
not started · partial · 100%
rolled back · permanent control · unknown
- **Cleanup state** – what work remains
active · stale · dead candidate · zombie · unknown

A flag can be **switched off**
and still evaluated **hundreds of times an hour**.
"Off" is not "unused". "Active" is not "needed".

Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.

But intent does not tell you state.

the two axes – what live evaluations and cleanup show later

A flag has **two independent axes**:

- **Rollout state** – what is being served

not started · partial · 100%
rolled back · permanent control · unknown

- **Cleanup state** – what work remains

active · stale · dead candidate · zombie · unknown

A flag can be **switched off**
and still evaluated **hundreds of times an hour**.
"Off" is not "unused". "Active" is not "needed".

One label per flag cannot carry both axes – and the label is what people reach for when deciding to delete.

Step 2 – Classify on two axes, not one label

By what it was created for:

the one label – picked once, close to the creation templates

1. **Release** – short-lived, should die at 100% rollout
2. **Experiment** – bounded by the experiment's end date
3. **Kill switch** – ops control, meant to be long-lived
4. **Permission / entitlement** – long-lived by design

Debt comes almost entirely from treating 1 and 2 like 3 and 4.

But intent does not tell you state.

the two axes – what live evaluations and cleanup show later

A flag has **two independent axes**:

- **Rollout state** – what is being served
not started · partial · 100%
rolled back · permanent control · unknown
- **Cleanup state** – what work remains
active · stale · dead candidate · zombie · unknown

A flag can be **switched off**
and still evaluated **hundreds of times an hour**.
"Off" is not "unused". "Active" is not "needed".

One label per flag cannot carry both axes – and the label is what people reach for when deciding to delete.

Your labels will differ by business, platform and environment. Keep two axes, and keep "unknown".

Demo – same badge, different truth

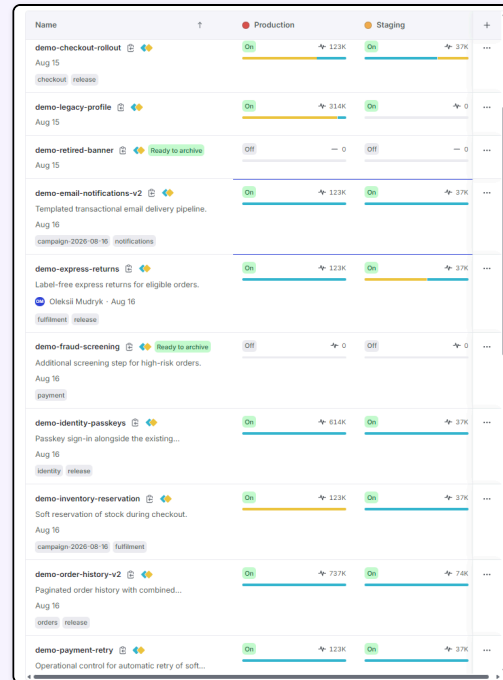
- "Ready to archive": the flag went quiet – its code may not have
- One badge: gone · still in code · "I don't know yet"
- Delete it while code still calls it, and the fallback answers

Silence you can trust is earned:
take the code out first, then the flag.

A company founded for this talk:
a free GitHub organisation and a free LaunchDarkly developer project.
Every key, name and number here is synthetic.

scenarios · github.com/moudrick/flag-lifecycle-sandbox (MIT)

demo org · github.com/featureflag-extensiveconsumer-demo-org (throw-away)



Name	Production	Staging	
demo-checkout-rollout	On 123K	On 37K	
demo-legacy-profile	On 31K	On 0	
demo-retired-banner	Ready to archive 0	Off 0	
demo-email-notifications-v2	On 123K	On 37K	
demo-express-returns	On 123K	On 37K	
demo-fraud-screening	Ready to archive 0	Off 0	
demo-identity-passkeys	On 614K	On 37K	
demo-inventory-reservation	On 123K	On 37K	
demo-order-history-v2	On 737K	On 74K	
demo-payment-retry	On 123K	On 37K	

Step 3 – Make Removal Cheap



Step 3 – Make Removal Cheap



- Let tooling strip dead code paths:
codemods, lint rules
an AI assistant that sees the whole codebase

Step 3 – Make Removal Cheap



- Let tooling strip dead code paths:
codemods, lint rules
an AI assistant that sees the whole codebase
- One PR per flag, small and mechanical, easy to review
PR checklist:
references gone · keeps the last active code path
rollback ready · old versions drained

Step 3 – Make Removal Cheap



- Let tooling strip dead code paths:
 - codemods, lint rules
 - an AI assistant that sees the whole codebase
- One PR per flag, small and mechanical, easy to review
 - PR checklist:
 - references gone · keeps the last active code path
 - rollback ready · old versions drained
- Track "flags removed" as a real, visible metric –
 - not just "flags shipped"

Step 3 – Make Removal Cheap



- Let tooling strip dead code paths:
codemods, lint rules
an AI assistant that sees the whole codebase
- One PR per flag, small and mechanical, easy to review
PR checklist:
references gone · keeps the last active code path
rollback ready · old versions drained
- Track "flags removed" as a real, visible metric –
not just "flags shipped"

Order matters: deprecate · remove the code · deploy · archive · delete last, if ever

Step 4 – Governance Without Bureaucracy

- Lightweight: an owner, an expiry, a Slack nudge
not a review board
- Quarterly flag review: 15 minutes
anything unowned or expired gets an owner and a next step, not a bulk delete
- Treat flag debt like any other tech debt
budget time for it explicitly
- A flag that turns up in an incident timeline
earns a cleanup ticket, not a name in a report

What *Not* To Do

- **Don't ban feature flags**

the safety net is real and valuable

- **Don't build governance first**

a naming convention and an owner field come before any platform

- **Don't clean up everything at once**

oldest, highest-risk flags first, in small monitored waves

- **Don't treat "Ready to archive" as a verdict**

it reads silence, not code



Takeaways

1. **Flags are a safety net *and* a liability**

plan for both

2. **Build the inventory from independent evidence**

not from one dashboard

3. **Classify on two axes**

and keep "I don't know yet" a legal answer

4. **Make removal as cheap as creation:**

code first, archive later, delete last

5. **Keep it flat with light governance:**

an owner, an expiry, small monitored waves



Thank you

Questions?

Oleksii Mudryk – moudrick.net

Experienced Platform/DevOps contractor

Principal DevOps Engineer at Talmatic 



download slides

moudrick.net/talks/ff-cleanup

